

Análisis de Resultados Reto 4

Grupo 4.

Camilo Romero – 202411065 - ca.romeror123@uniandes.edu.co

Mariana Vélez – 202123931 – m.velezr@uniandes.edu.co

En este documento se analizará la complejidad temporal de los algoritmos desarrollados para la resolución del Reto 4 de la asignatura Estructuras de Datos y Algoritmos. Para este propósito, se utilizó la función time de Python, realizando pruebas de rendimiento con archivos CSV, comenzando desde el menos pesado hasta el más pesado.

Requerimiento 1.

Estudiante encargado: Camilo Romero

Para evaluar la eficiencia temporal de este algoritmo se utilizó como valor de entrada constante:

Digite el primer vértice: 7073

Digite el segundo vértice: 9830

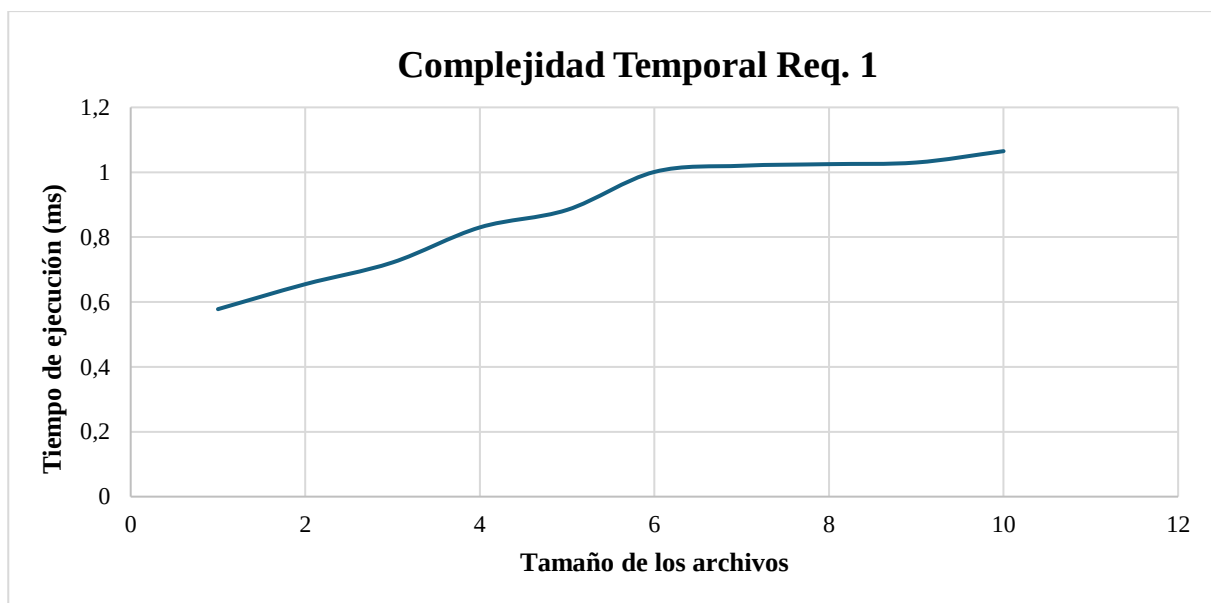
Tabla 1. Tiempo de Ejecución Requerimiento 1

Archivo	Tiempo de ejecución (ms)
_10.csv	0.578
_20.csv	0.655
_30.csv	0.722
_40.csv	0.830
_50.csv	0.884
_60.csv	1.001
_70.csv	1.020
_80.csv	1.025
_90.csv	1.030
_large.csv	1.065

Este requerimiento tiene como objetivo identificar, dentro de un grafo, si existe una conexión entre dos usuarios. Para ello, se implementó el algoritmo de Búsqueda en Profundidad (DFS) en los vértices de relaciones. A pesar de involucrar varias funciones recursivas adicionales, la complejidad temporal predominante en este requisito está determinada por el tamaño del grafo, denotado como $O(n)$, donde n es la cantidad de nodos. El algoritmo DFS recorre la mayoría de los nodos del grafo para determinar la relación entre los dos usuarios. Tras implementar la solución, se llevaron a cabo pruebas que permitieron graficar la relación entre

el tiempo de ejecución (en milisegundos) y el tamaño del archivo procesado, evaluando así el rendimiento.

Gráfica 1. Complejidad Temporal Requerimiento 1



La gráfica revela que el comportamiento del algoritmo sigue una tendencia ligeramente lineal, lo que indica que el crecimiento temporal está relacionado con la cantidad de datos que procesa. Sin embargo, se observa cierta tendencia a mantenerse constante en el tiempo a pesar de incrementar los datos de entrada. Esto podría atribuirse a que, aunque el tamaño de los archivos aumenta, la cantidad de información relevante para el valor de entrada no necesariamente crece de manera proporcional, lo que genera variaciones en el tiempo de ejecución.

Requerimiento 3.

Estudiante encargado: Mariana Vélez

Para evaluar la eficiencia temporal de este algoritmo se utilizó como valor de entrada constante:

Ingrese ID de la persona: 7073

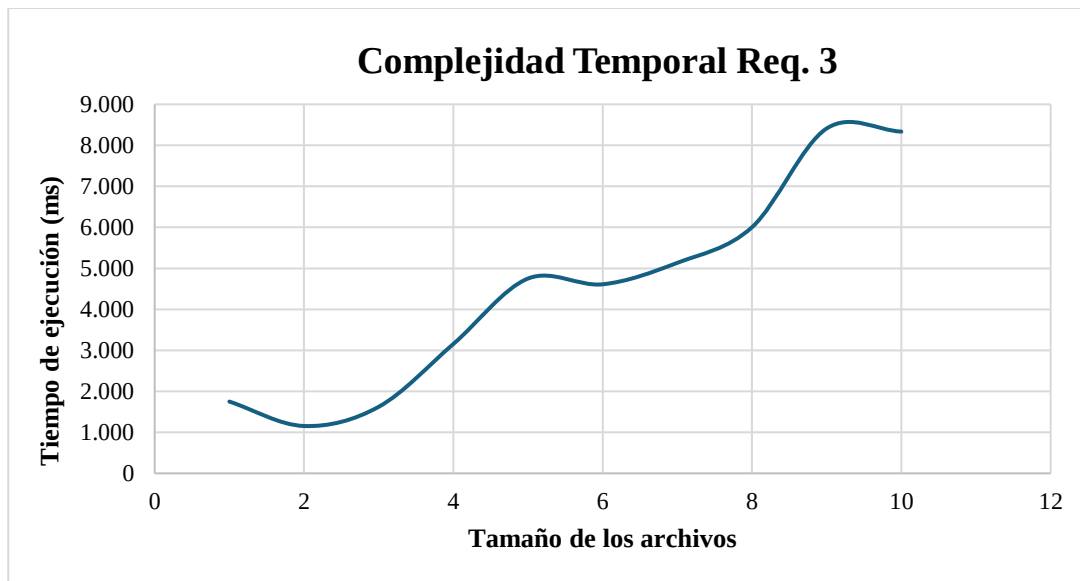
Tabla 2. Tiempo de Ejecución Requerimiento 3

Archivo	Tiempo de ejecución (ms)
_10.csv	1.751
_20.csv	1.155

_30.csv	1.623
_40.csv	3.155
_50.csv	4.751
_60.csv	4.611
_70.csv	5.135
_80.csv	6.003
_90.csv	8.411
_large.csv	8.335

Este requerimiento busca identificar, dentro de un grafo, al amigo del ID ingresado como parámetro que tiene la mayor cantidad de seguidores. Para lograrlo, la función primero obtiene la lista de seguidores del ID dado, luego cuenta los seguidores de cada uno de estos amigos y finalmente compara las cantidades para determinar el mayor. Dado que se realizan tres iteraciones principales sobre la lista de seguidores (obtenerlos, contar seguidores y comparar), la función tiene una complejidad esperada de $O(n^3)$. Tras implementar la solución, se llevaron a cabo pruebas que permitieron graficar la relación entre el tiempo de ejecución (en milisegundos) y el tamaño del archivo procesado, evaluando así el rendimiento.

Gráfica 2. Complejidad Temporal Requerimiento 3



La gráfica revela que el comportamiento del algoritmo sigue una tendencia ligeramente exponencial en ciertos tramos, lo que indica que el crecimiento temporal está relacionado con la cantidad de datos que procesa. Sin embargo, se observan picos aislados que interrumpen esta tendencia. Esto podría atribuirse a que, aunque el tamaño de los archivos aumenta, la cantidad de información relevante para el valor de entrada no necesariamente crece de manera proporcional, lo que genera variaciones en el tiempo de ejecución.

Requerimiento 4.

Estudiante encargado: Camilo Romero

Para evaluar la eficiencia temporal de este algoritmo se utilizó como valor de entrada constante:

Digite el primer ID: 7073

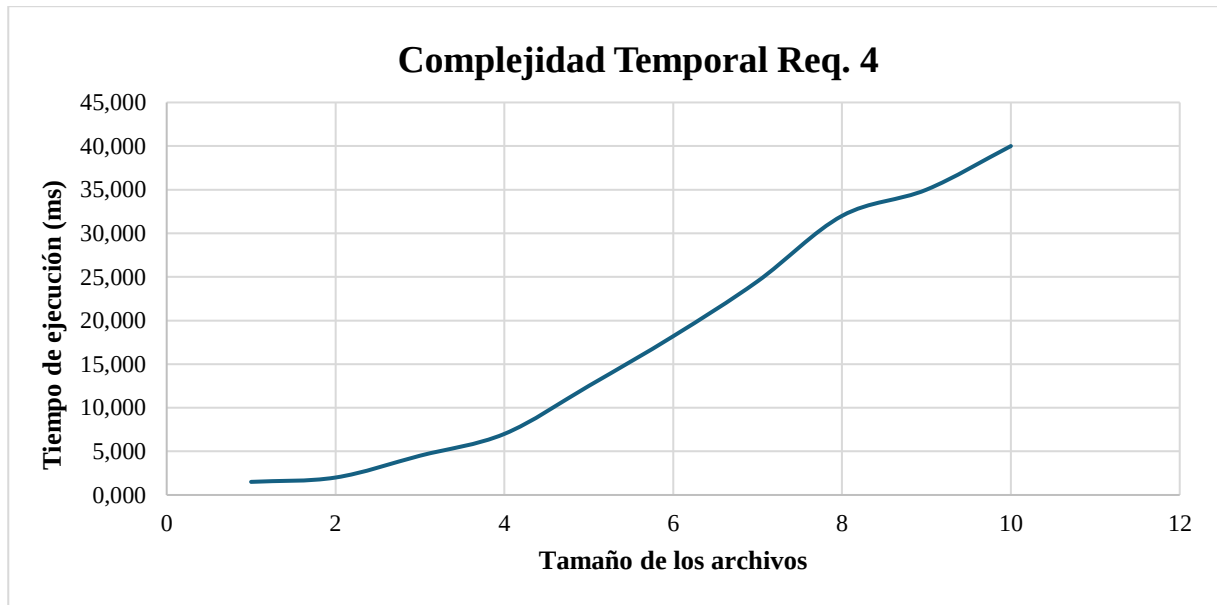
Digite el segundo ID: 9830

Tabla 3. Tiempo de Ejecución Requerimiento 4

Archivo	Tiempo de ejecución (ms)
_10.csv	1.500
_20.csv	2.000
_30.csv	4.504
_40.csv	7.001
_50.csv	12.503
_60.csv	15.335
_70.csv	24.500
_80.csv	32.000
_90.csv	35.004
_large.csv	40.000

Este requerimiento busca identificar, dentro de un grafo, los amigos que un usuario A comparte con un usuario B. Para lograrlo, la función primero obtiene la lista de seguidores del usuario A, y repite el mismo procedimiento para el usuario B. Para determinar que amigos comparten se debe iterar sobre ambas listas para comparar, esto le otorga una complejidad temporal de $O(n^2)$. Tras implementar la solución, se llevaron a cabo pruebas que permitieron graficar la relación entre el tiempo de ejecución (en milisegundos) y el tamaño del archivo procesado, evaluando así el rendimiento.

Gráfica 3. Complejidad Temporal Requerimiento 4



La gráfica revela que el comportamiento del algoritmo sigue una tendencia ligeramente exponencial, lo que indica que el crecimiento temporal está relacionado con la cantidad de datos que procesa. Sin embargo, la complejidad temporal no incrementa en la misma proporción según el tamaño del archivo. Esto podría atribuirse a que, aunque el tamaño de los archivos aumenta, la cantidad de información relevante para el valor de entrada no necesariamente crece de manera proporcional, lo que genera variaciones en el tiempo de ejecución.

Requerimiento 6.

Estudiante encargado: Camilo Romero

Para evaluar la eficiencia temporal de este algoritmo se utilizó como valor de entrada constante:

Digite cuantas cuentas quiere: 4

Tabla 4. Tiempo de Ejecución Requerimiento 6

Archivo	Tiempo de ejecución (ms)
_10.csv	1.675
_20.csv	1.980
_30.csv	4.504
_40.csv	7.001
_50.csv	11.667
_60.csv	15.304
_70.csv	21.089
_80.csv	21.003

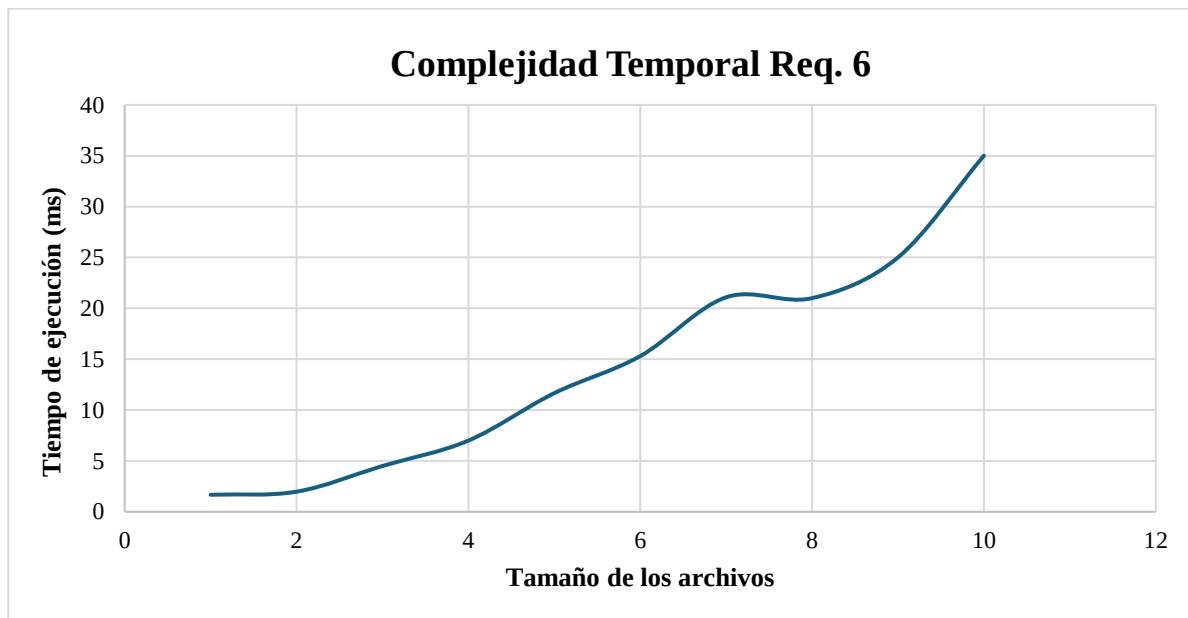
_90.csv	25.007
_large.csv	35.008

Este requerimiento tiene como objetivo identificar a los N usuarios más populares, es decir los que más seguidores tienen con el propósito de definir si existe un árbol que los conecte.

En términos de complejidad, n representa el tamaño total del archivo de relaciones y seguidores. La función recorre todo el grafo de seguidores y llama al algoritmo DFS para recorrer el grafo de relaciones, por ende, se espera una complejidad temporal de $O(n^2)$.

Tras realizar las respectivas pruebas, se procede a graficar los valores del Tiempo de ejecución (ms) vs tamaño del Archivo.

Gráfica 4. Complejidad Temporal Requerimiento 6.



La gráfica revela que el comportamiento del algoritmo sigue una tendencia ligeramente exponencial, lo que indica que el crecimiento temporal está relacionado con la cantidad de datos que procesa. Sin embargo, la complejidad temporal no incrementa en la misma proporción según el tamaño del archivo. Esto podría atribuirse a que, aunque el tamaño de los archivos aumenta, la cantidad de información relevante para el valor de entrada no necesariamente crece de manera proporcional, lo que genera variaciones en el tiempo de ejecución.

Requerimiento 7.

Estudiante encargado: Mariana Vélez

Para evaluar la eficiencia temporal de este algoritmo se utilizó como valor de entrada constante:

Digite los Hobbies que desea buscar: crossfit,lucha

Ingrese ID de la persona: 4685

Tabla 5. Tiempo de Ejecución Requerimiento 7

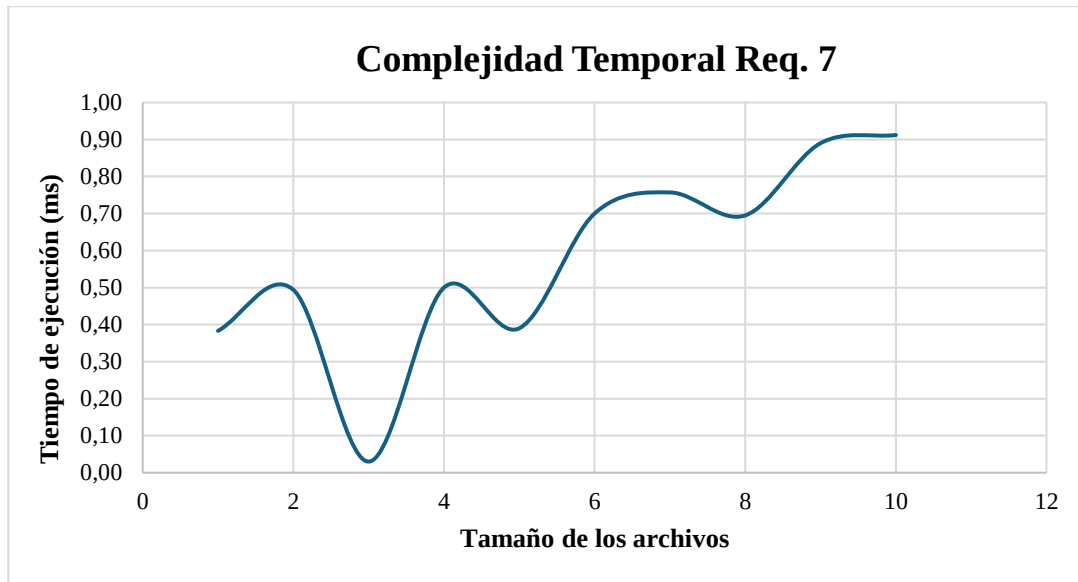
Archivo	Tiempo de ejecución (ms)
_10.csv	0.383
_20.csv	0.494
_30.csv	0.030
_40.csv	0.500
_50.csv	0.390
_60.csv	0.700
_70.csv	0.757
_80.csv	0.695
_90.csv	0.890
_large.csv	0.912

Este requerimiento tiene como objetivo identificar a todos los seguidores del ID proporcionado como parámetro que compartan al menos uno de los hobbies también ingresados como parámetro. Si la conexión es directa, se considera un nivel de profundidad de 1, pero si la relación es indirecta (es decir, a través de un amigo del seguidor), el nivel de profundidad aumenta. El proceso comienza obteniendo todos los seguidores del ID inicial y verificando cuáles comparten al menos un hobby. Posteriormente, se toman esos seguidores y se identifican sus respectivos seguidores, evaluando si comparten algún hobby, lo que incrementa el nivel de profundidad. Este análisis se realiza utilizando una función auxiliar ("máscara") que evalúa las relaciones con nivel de profundidad 1 y luego llama a una función recursiva para analizar niveles de profundidad mayores, usando los IDs de los seguidores identificados previamente.

En términos de complejidad, m representa el tamaño total del archivo de usuarios, ya que este valor determina la condición de parada de la función recursiva. Por otro lado, n corresponde al tamaño de las listas de seguidores, mientras que a representa la cantidad de hobbies evaluados. Dado que la recursión y las iteraciones se realizan sobre estas dimensiones, se espera que la complejidad temporal de la función sea $O(n^4 \cdot a^4 \cdot m)$.

Tras realizar las respectivas pruebas, se procede a graficar los valores del Tiempo de ejecución (ms) vs tamaño del Archivo.

Gráfica 5. Complejidad Temporal Requerimiento 7.



La gráfica muestra que el comportamiento del algoritmo no sigue una tendencia específica, sino que es altamente variable. Esto sugiere que, aunque el tamaño de los archivos aumenta, la cantidad de información relevante asociada al parámetro de entrada no crece de manera proporcional. En particular, se observa que el archivo `_30.csv` presenta el menor tiempo de ejecución, lo cual se debe a que no se identifican seguidores que cumplan con los requisitos establecidos, reduciendo así el tiempo de procesamiento de manera significativa.